

CRUX: HOW TO HANDLE COMMON CONCURRENCY BUGS

Concurrency bugs tend to come in a variety of common patterns. Knowing which ones to look out for is the first step to writing more robust, correct concurrent code.

What types of bugs exist?

The study (Zu et al [L+08]) analyzed four open-source applications.

They found & fixed 105 concurrency bugs.

Application	What it does	Non-Deadlock	Deadlock
MySQL	Database Server	14	9
Apache	Web Server	13	4
Mozilla	Web Browser	41	16
OpenOffice	Office Suite	6	2
Total		74	31

Non-deadlock bugs

Non-deadlock bugs make up a majority of concurrency bugs.

Atomicity-Violation Bugs

Example from MySQL:

```
1 Thread 1::
2 if (thd->proc_info) {
3     fputs(thd->proc_info, ...);
4 }
5
6 Thread 2::
7 thd->proc_info = NULL;
```

In the example, two different threads access the field `proc_info` in

the structure `thd`. The first thread checks if the value is non-NULL and then prints its value; the second thread sets it to NULL. Clearly, if the first thread performs the check but then is interrupted before the call to `fputs`, the second thread could run in-between, thus setting the pointer to NULL; when the first thread resumes, it will crash, as a NULL pointer will be dereferenced by `fputs`.

Formal definition of an atomicity violation:

"The desired serializability among multiple memory accesses is violated i.e. a code region is intended to be atomic, but atomicity is not enforced during execution."

Simply, add locks around shared variable references.

```
1 pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Thread 1::
4 pthread_mutex_lock(&proc_info_lock);
5 if (thd->proc_info) {
6     fputs(thd->proc_info, ...);
7 }
8 pthread_mutex_unlock(&proc_info_lock);
9
10 Thread 2::
11 pthread_mutex_lock(&proc_info_lock);
12 thd->proc_info = NULL;
13 pthread_mutex_unlock(&proc_info_lock);
```

Order-Violation Bugs

```
1 Thread 1::
2 void init() {
3     mThread = PR_CreateThread(mMain, ...);
4 }
5
6 Thread 2::
7 void mMain(...) {
8     mState = mThread->State;
9 }
```

The code in Thread 2 seems to assume that the variable `mThread` has already been initialized and is NOT NULL.

However, if Thread 2 runs immediately once created,

the value of mThread will not be set when it is accessed within mMain() & will crash.

Formal definition of an order violation bug:

"The desired order between two (groups of) memory accesses is flipped i.e A should always be executed before B, but order is not enforced during execution"

The fix is to enforce ordering via condition variables.

```
1 pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3 int mtInit
4     = 0;
5
6 Thread 1::
7 void init() {
8     ...
9     mThread = PR_CreateThread(mMain, ...);
10
11    // signal that the thread has been created...
12    pthread_mutex_lock(&mtLock);
13    mtInit = 1;
14    pthread_cond_signal(&mtCond);
15    pthread_mutex_unlock(&mtLock);
16    ...
17 }
18
19 Thread 2::
20 void mMain(...) {
21     ...
22    // wait for the thread to be initialized...
23    pthread_mutex_lock(&mtLock);
24    while (mtInit == 0)
25        pthread_cond_wait(&mtCond, &mtLock);
26    pthread_mutex_unlock(&mtLock);
27
28    mState = mThread->State;
29    ...
30 }
```

In this fixed-up code sequence, we have added a condition variable (mtCond) and corresponding lock (mtLock), as well as a state variable (mtInit). When the initialization code runs, it sets the state of mtInit to 1 and signals that it has done so. If Thread 2 had run before this point, it will be waiting for this signal and corresponding state change; if it runs

later, it will check the state and see that the initialization has already occurred (i.e., `mtInit` is set to 1), and thus continue as is proper. Note that we could likely use `mThread` as the state variable itself, but do not do so for the sake of simplicity here. When ordering matters between threads, condition variables (or semaphores) can come to the rescue.

Deadlock Bugs:

A deadlock occurs when a thread T_1 is holding a lock L_1 and waiting for another one (L_2).

Unfortunately, thread T_2 holds lock L_2 & is waiting for lock L_1 to be released.

In code, this is:

Thread 1:

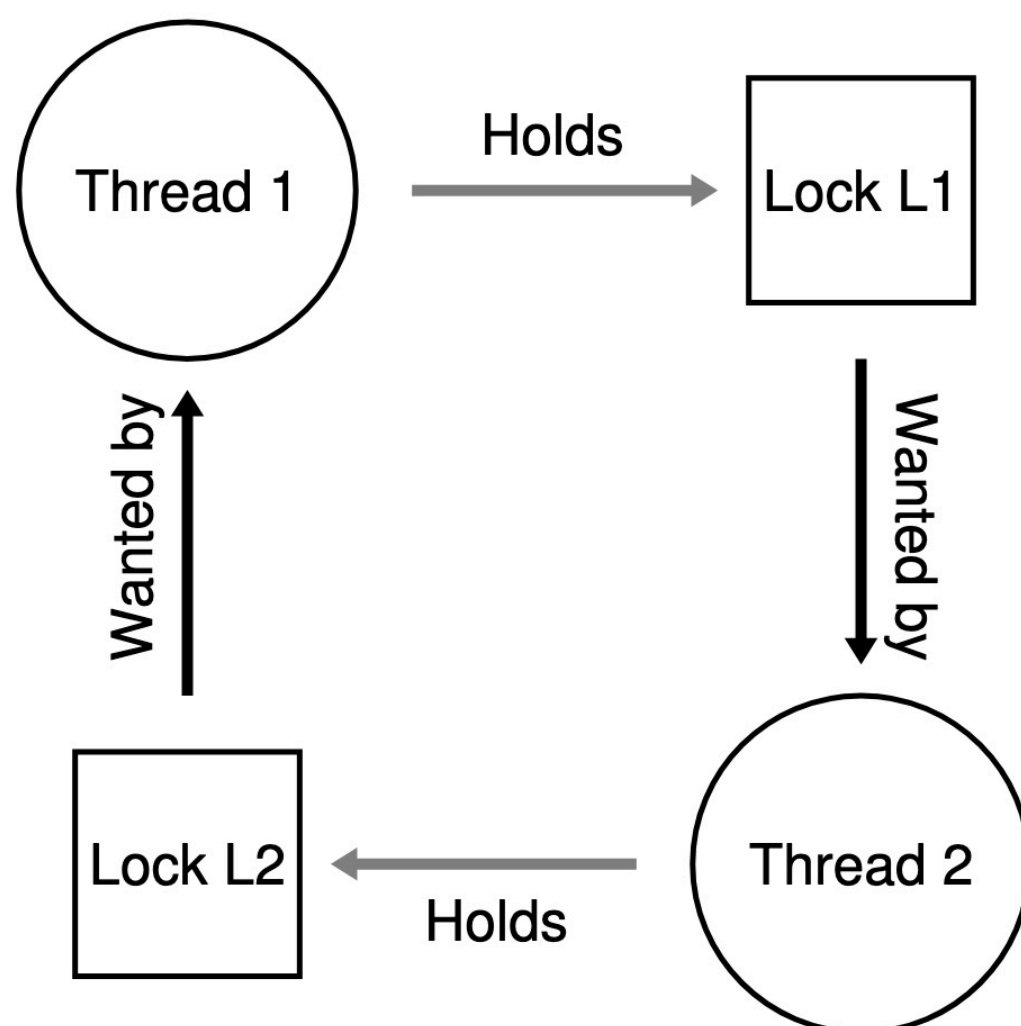
```
pthread_mutex_lock(L1);  
pthread_mutex_lock(L2);
```

Thread 2:

```
pthread_mutex_lock(L2);  
pthread_mutex_lock(L1);
```

If this code runs, deadlock does not necessarily occur; rather it may occur.

For example,



The presence of a cycle is indicative of the deadlock.

CRUX: HOW TO DEAL WITH DEADLOCK

How should we build systems to prevent, avoid, or at least detect and recover from deadlock? Is this a real problem in systems today?

Why Do Deadlocks Occur?

In above example, if both threads made sure to grab locks in the same order, the deadlock would never arise.

So, why do deadlocks arise?

1. In large code bases, complex dependencies arise.

For example, in OS,

Virtual memory asks for a data from hard drive, it locks its resources. So, no changes occur while it is working.

The file system receives the request & it also locks its own resources (I/O) & calls VM for space to read the data into.

VM is waiting on FS & FS on VM.

2. As programmers, we are taught to hide details of implementation and thus build in a modular way i.e encapsulation.

For example, take java vector class & the method `AddAll()`.

```
Vector v1, v2;  
v1.AddAll(v2);
```

Internally, locks for both vectors being added needs to be acquired.

They acquire locks in some arbitrary order (v_1 before v_2).

If some other thread calls $v_2.AddAll(v_1)$ at nearly the same time, we have potential for deadlock.

Conditions for Deadlock

Four conditions need to hold for deadlock to occur.

- **Mutual exclusion:** Threads claim exclusive control of resources that they require (e.g., a thread grabs a lock).
- **Hold-and-wait:** Threads hold resources allocated to them (e.g., locks that they have already acquired) while waiting for additional resources (e.g., locks that they wish to acquire).
- **No preemption:** Resources (e.g., locks) cannot be forcibly removed from threads that are holding them.
- **Circular wait:** There exists a circular chain of threads such that each thread holds one or more resources (e.g., locks) that are being requested by the next thread in the chain.

If any of these four conditions are not met, deadlock cannot occur.

Prevention

Circular Wait

Write your locking code in way such that you never induce a circular wait.

The most straightforward way to do this is to provide a total ordering on lock acquisition.

For example, if there are only two locks (L_1 & L_2), you can prevent deadlock by always acquiring L_1 before L_2 .

→ Acquire a global hierarchy.
Acquire lower no lock before higher.

In complex systems, total ordering is difficult to achieve. Thus, partial ordering can be useful.

For example,
In Linux memory mapping code, the comment at top reveals ten different groups of lock acquisition order.

“i_mutex before i_mmap_rwsem” and more complex orders such as “i_mmap_rwsem before private_lock before swap_lock before i_pages lock”.

TIP: ENFORCE LOCK ORDERING BY LOCK ADDRESS

In some cases, a function must grab two (or more) locks; thus, we know we must be careful or deadlock could arise. Imagine a function that is called as follows: `do_something(mutex_t *m1, mutex_t *m2)`. If the code always grabs `m1` before `m2` (or always `m2` before `m1`), it could deadlock, because one thread could call `do_something(L1, L2)` while another thread could call `do_something(L2, L1)`.

To avoid this particular issue, the clever programmer can use the *address* of each lock as a way of ordering lock acquisition. By acquiring locks in either high-to-low or low-to-high address order, `do_something()` can guarantee that it always acquires locks in the same order, regardless of which order they are passed in. The code would look something like this:

```
if (m1 > m2) { // grab in high-to-low address order
    pthread_mutex_lock(m1);
    pthread_mutex_lock(m2);
} else {
    pthread_mutex_lock(m2);
    pthread_mutex_lock(m1);
}
// Code assumes that m1 != m2 (not the same lock)
```

By using this simple technique, a programmer can ensure a simple and efficient deadlock-free implementation of multi-lock acquisition.

Hold-and-wait

The hold-and-wait requirement can be avoided by acquiring all locks at once.

```
1  pthread_mutex_lock(prevention); // begin acquisition
2  pthread_mutex_lock(L1);
3  pthread_mutex_lock(L2);
4  ...
5  pthread_mutex_unlock(prevention); // end
```

By first grabbing the lock `prevention`, this code guarantees that no untimely thread switch can occur in the midst of lock acquisition and thus

deadlock can be avoided.

Now, threads can acquire locks (L_1 & L_2) in different order as well.

- Encapsulation works against us: when calling a routine, this approach requires us to know exactly which locks must be held & to acquire them ahead of time.
- It decreases concurrency, as we acquire all locks & not only when we need them.

Imagine a function that does math for 9 seconds & in last second writes to a shared file.

This approach would require us to hold the lock for 10 seconds (from start of function to end).

No Preemption

The routine `pthread_mutex_trylock()` either grabs the lock (if available) and returns success or returns an error indicating the lock is held instead of sleeping.

Thus, you try again later.

```
1  top:
2      pthread_mutex_lock(L1);
3      if (pthread_mutex_trylock(L2) != 0) {
4          pthread_mutex_unlock(L1);
5          goto top;
6      }
```

This would work as a good solution for java vector add problem

Another thread could follow the same protocol but grab the locks in the other order (L_2 then L_1) and the program would be deadlock free.

- Livelock: it is possible that two threads could both be repeatedly attempting this sequence

and repeatedly failing. In this case, the system is running but no progress is being made.

A solution to this problem is to add a random delay before trying again.

- If one of these locks is buried in some routine that is getting called & the code had acquired some other resources along the way, it must make sure to carefully release them as well.

For example,
if after acquiring lock L_1 , the code allocated some memory, it would have to be released upon failure to acquire L_2 .

Mutual Exclusion

The final technique is to remove the need for mutual exclusion.

We can design various data structures without locks at all by using hardware instructions.

Imagine we wanted to atomically increment a value by certain amount. We could do so without locks as:

```
1 void AtomicIncrement(int *value, int amount) {  
2     do {  
3         int old = *value;  
4     } while (CompareAndSwap(value, old, old + amount) == 0);  
5 }
```

There is ^{no} deadlock but livelock is a possibility.

Imagine a more complex example: list insertion. We can solve this using traditional locking.

```
1 void insert(int value) {  
2     node_t *n = malloc(sizeof(node_t));  
3     assert(n != NULL);  
4     n->value = value;
```



```

5 pthread_mutex_lock(listlock); // begin critical section
6 n->next = head;
7 head = n;
8 pthread_mutex_unlock(listlock); // end critical section
9 }

```

²The astute reader might be asking why we grabbed the lock so late, instead of right when entering `insert()`; can you, astute reader, figure out why that is likely correct? What assumptions does the code make, for example, about the call to `malloc()`?

Allocating & setting value (lines 2-4) only affect local variable `n`.

Reading & updating head pointer (lines 6-7) affect global variable `head`. So, we need a lock.

Lastly, the code assumes `malloc` is thread-safe i.e. no race condition inside it.

Instead, let's do it in lock free manner

```

1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     do {
6         n->next = head;
7     } while (CompareAndSwap(&head, n->next, n) == 0);
8 }

```

The code here updates the next pointer to point to the current head, and then tries to swap the newly-created node into position as the new head of the list. However, this will fail if some other thread successfully swapped in a new head in the meanwhile, causing this thread to retry again with the new head.

Deadlock Avoidance via Scheduling

Avoidance requires some global knowledge of which locks various threads might grab during their execution, and subsequently schedules said threads in a way as to guarantee no deadlock can occur.

For example,

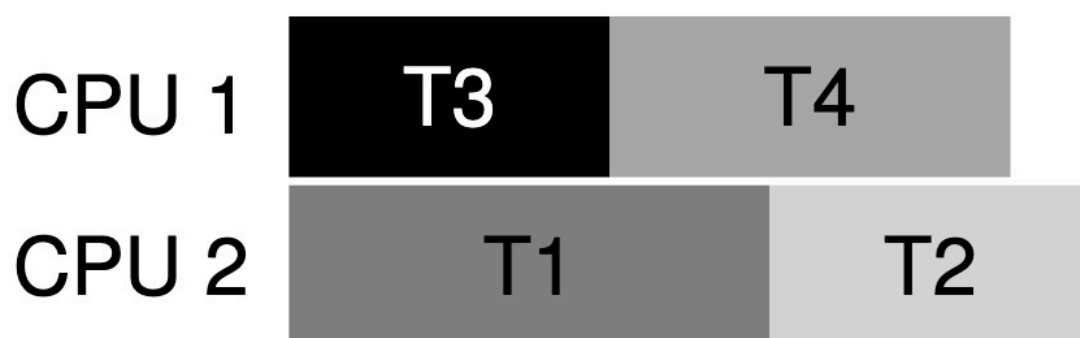
Assume we have 2 processors & four threads

to be scheduled.

Further, we know that thread T_1 grabs locks L_1 & L_2 , T_2 grabs L_1 & L_2 as well, T_3 grabs L_2 only & T_4 grabs only.

	T1	T2	T3	T4
L1	yes	yes	no	no
L2	yes	yes	yes	no

We could schedule them as to avoid deadlocks i.e. as long as T_1 & T_2 don't run at same time, there is no deadlock.

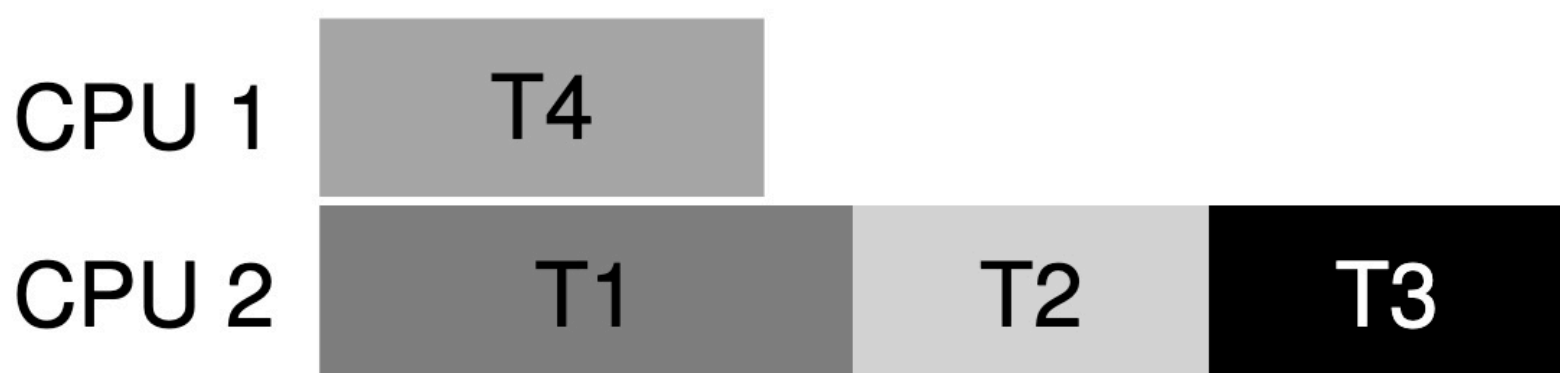


Note that it is OK for T_3 & T_1 or T_3 & T_2 to overlap. Even T_3 grabs L_2 , it can never cause a deadlock as it only grabs one lock.

Another example, threads T_1 - T_3 need grab both locks.

	T1	T2	T3	T4
L1	yes	yes	yes	no
L2	yes	yes	yes	no

Below is a possible schedule



Now, T_1 - T_3 run on same processor, and total time to complete jobs has lengthened.

These approaches are useful in an embedded environment where one has full knowledge of entire set of tasks.

Detect & Recover

One final strategy is to allow deadlocks to occasionally occur, and then take some action once such a deadlock has been detected.

Many database systems employ deadlock detection and recovery techniques. A deadlock detector runs periodically, building a resource graph and checking it for cycles. In the event of a cycle (deadlock), the system needs to be restarted. If more intricate repair of data structures is first required, a human being may be involved to ease the process.